

NATIONAL ECONOMICS UNIVERSITY



**PROJECT
DATABASE MANAGEMENT SYSTEM**

SQLLedger

**PERSONAL FINANCE
MANAGEMENT SYSTEM**

Student: Hoang Linh Phuong (11245925)

Instructor: Dr. Tran Hung

Ha Noi, 2026

PROJECT 13: PERSONAL FINANCE MANAGEMENT SYSTEM

DATCOM Lab
NEU-College of Technology
National Economics University
Email: hung.tran@neu.edu.vn

Project Objective

The objective of this project is to develop a system that enables users to effectively manage personal income and expenses, track financial activities, analyze spending habits, and optimize savings through data-driven reports and alerts.

Database and Programming Languages

- **Database Management System:** MySQL
- **Programming Language:** Python

Detailed Project Requirements

System Analysis and Requirements

- Manage personal financial data including income, expenses, bank accounts, and spending categories.
- Allow users to monitor their financial health through detailed summaries and visual reports.

Main Functionalities

- User profile management.
- Income and expense entry, update, and categorization.
- Bank account tracking and balance history.
- Daily, monthly, and yearly financial summaries.
- Budget planning and spending limit alerts.
- Reporting on trends and category-wise expenditures.

Database Design and Implementation

1. Data Model Design

- Create an ER diagram modeling users, income, expenses, and accounts.
- Convert to relational schema with PKs, FKs, and necessary constraints.

2. Table Structures

- Users (UserID, UserName, Email, PhoneNumber)
- Income (IncomeID, UserID, Amount, IncomeDate, Description)
- ExpenseCategories (CategoryID, CategoryName)
- Expenses (ExpenseID, UserID, CategoryID, Amount, ExpenseDate, Description)
- BankAccounts (AccountID, UserID, BankName, Balance)

3. Sample Data

- Populate each table with 5–10 representative records.
- Create a database schema diagram using MySQL Workbench.

Advanced Database Objects

- **Indexes:** Enhance performance for user queries and expense lookups.
- **Views:** Create monthly income/expense summaries, and category-wise spending views.
- **Stored Procedures:** Automate balance updates and monthly closure operations.
- **User Defined Functions:** Compute total income, total expenses, or budget status.
- **Triggers:** Automatically update bank account balance upon income or expense entry.

Database Security and Administration

- Secure user information with authentication roles.
- Implement access controls to protect personal financial data.
- Ensure backup and recovery procedures are in place.
- Optimize query performance for report generation.

Python Application Development

- **Database Connection:** Use `mysql-connector-python` or `SQLAlchemy`.
- **Finance Tracker:** Build scripts for users to input transactions, track expenses, and view summaries.
- **Reporting:** Generate graphical and tabular financial reports.
- **User Interface:** Optional CLI or GUI to enhance user experience.

Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Summarize key achievements of the system.
- Recommend features such as saving goals, predictive analytics, or integration with mobile banking.

References

- Include all resources, documentation, and libraries used during the project.

Abstract

This report presents **SQLLedger**, a comprehensive Personal Finance Management System designed to help individuals automate the tracking of their income, expenses, and overall financial health. Built upon a two-tier architecture, the project leverages a robust **MySQL** database for back-end data integrity and a **Python (Tkinter)** desktop application for an intuitive graphical user interface.

The core of the system relies heavily on advanced relational database concepts. Data consistency and automation are strictly enforced at the database level using **Stored Procedures** for transaction validation, **Triggers** for real-time bank balance updates and audit logging, and **Views** coupled with **User-Defined Functions (UDFs)** for dynamic financial reporting and budget status alerts. Additionally, the application integrates **Matplotlib** to provide interactive visual analytics and employs **bcrypt** for secure, role-based user authentication.

Ultimately, this project demonstrates the practical application of database management theories to solve real-world financial tracking challenges, delivering a reliable, automated, and secure system.

Contents

1	Project Overview	7
1.1	Project Objectives	7
1.2	Technologies Used	7
1.3	Project Links	8
1.4	Project Structure	8
1.5	System Architecture	9
1.6	Key Features	10
2	Database Analysis and Design	11
2.1	Data Model - ER Diagram	11
2.2	Relational Schema	12
2.3	Data Constraints	13
2.4	Sample Data	14
3	Advanced Database Implementation	17
3.1	Indexes	17
3.2	Views	18
3.3	User Defined Functions (UDFs)	19
3.4	Triggers	20
3.5	Stored Procedures	21
3.6	Database Security and Access Control	22
4	Python Application Development	24
4.1	Application Architecture	24
4.2	Database Connection Layer	25
4.3	Graphical User Interface (GUI)	26
4.4	Reporting and Data Visualization	32
5	Conclusion and Recommendation	34
5.1	Achievements and Key Deliverables	34
5.2	Future Developments	34

List of Figures

1	Entity-Relationship Diagram - Personal Finance Management System . .	11
2	SHOW CREATE TABLE Expenses - Foreign Key Constraints	13
3	Sample data - Users table (5 records)	14
4	Sample data - BankAccounts table (6 records)	14
5	Sample data - Income table (8 records)	14
6	Sample data - Expenses table (10 records)	15
7	Sample data - ExpenseCategories table	15
8	All indexes on the Expenses table confirmed via SHOW INDEX FROM Expenses LIMIT 5	17
9	Query result of MonthlyFinancialSummary for the sample dataset	18
10	Query result of CategoryWiseSpending for the sample dataset	19
11	SELECT GetBudgetStatus(1, 5, 2026) returning SURPLUS for User 1 in May 2026	20
12	BalanceAuditLog automatically populated after several income and ex- pense transactions	21
13	Result of calling CALL AddExpense(1, 1, 1500, '2026-05-13', 'Dinner') - procedure returns confirmation and remaining balance	22
14	Login window, email, and password authentication	27
15	Login window, email, and password authentication	27
16	Dashboard - stat cards, budget status, recent transactions, and bar chart	28
17	Income page - entry form and transaction history	29
18	Expenses page - category dropdown, entry form, and history table	30
19	Reports page - monthly bar chart and category spending pie chart	31
20	Accounts page - account balance cards and balance audit log	32
21	Monthly Income vs Expense bar chart embedded in the Reports page . .	32
22	Category spending pie chart embedded in the Reports page	33

List of Tables

1	Technologies and Libraries Used	8
2	Project Links and Resources	8
3	Project Directory Structure	9
4	System Architecture Layers	9
5	Key Features Implementation	10
6	Database Tables Description	11
7	Foreign Key Constraints and Behaviors	12
8	Data Integrity Constraints	13
9	Sample Dataset Summary	16
10	Database Indexes and Impact	17
11	User Defined Functions (UDFs) and Roles	19
12	Database Triggers and Roles	20
13	Database Security Users and Permissions	23
14	Python Application Architecture	24
15	Database Connection Layer Functions	25
16	Project Achievements Summary	34

1 Project Overview

1.1 Project Objectives

In today's fast-paced financial environment, individuals often struggle to maintain a clear picture of their income, expenses, and overall financial health. Manual tracking through spreadsheets or notebooks is error-prone and time-consuming. SQLLedger addresses this challenge by providing a structured, automated solution for personal financial management.

The core objectives of this system are:

- Enable users to record, categorize, and monitor all income and expense transactions in real time.
- Automatically update bank account balances upon every financial transaction using database triggers.
- Provide detailed financial summaries and visual analytics to help users understand their spending habits.
- Implement budget status tracking (SURPLUS / BALANCED / DEFICIT) to alert users of financial risk.
- Maintain a comprehensive audit trail of all balance changes for transparency and accountability.
- Secure user data through role-based access control and password hashing.
- Offer an intuitive dark-mode GUI application that requires no prior technical knowledge to operate.

1.2 Technologies Used

The system is built on a two-tier client-server architecture. The database layer is managed by MySQL, while the application layer is developed in Python.

The following table summarizes all technologies and libraries used throughout the project:

Table 1: Technologies and Libraries Used

Component	Technology / Library	Version	Purpose
Database	MySQL	8.4	Relational database management
Programming Language	Python	3.12	Application development
DB Connector	mysql-connector-python	9.3.0	Python-MySQL communication
GUI Framework	Tkinter	Built-in	Desktop graphical interface
Data Visualization	Matplotlib	3.10.8	Charts and graphs
Password Security	bcrypt	5.0.0	Password hashing and verification
Cloud Database	Aiven (MySQL)	8.4	Online database hosting
Packaging	PyInstaller	6.20.0	Packages the application into a standalone executable file

1.3 Project Links

The complete source code and a demonstration video are publicly available at the following links:

Table 2: Project Links and Resources

Resource	URL
GitHub Repository	https://github.com/alexhoang1110/sqlledger-personal-financial-management-system/
YouTube Demo Video	https://youtu.be/H04tPitm03I

1.4 Project Structure

The project is organized into two main directories: `sql/` for all database-related scripts, and `python/` for the application source code. This separation ensures modularity and makes it easy to set up the database independently of the application.

Table 3: Project Directory Structure

Directory/File	Description
sql/01_schema.sql	Database and table creation with constraints
sql/02_indexes.sql	Performance indexes on key columns
sql/03_views.sql	Monthly summary and category spending views
sql/04_functions.sql	UDFs: GetTotalIncome, GetTotalExpenses, GetBudgetStatus
sql/05_triggers.sql	Auto-update balance and audit log triggers
sql/06_procedures.sql	Stored procedures: AddIncome, AddExpense, Monthly-Closure
sql/07_security.sql	Role-based user access control
sql/run_all.sql	Consolidated script to run all SQL files at once
python/main.py	Application entry point
python/config.py	Database connection configuration
python/database.py	Core database access functions
python/models/	Data access layer (user, income, expense, account)
python/gui/	GUI components (theme, pages, navigation)
python/utils/csv_handler.py	CSV import and export utilities
python/data/	Sample CSV data files for initial seeding
docs/ER_Diagram.png	Entity-Relationship diagram

1.5 System Architecture

The application follows a simplified MVC (Model-View-Controller) pattern, clearly separating data access, business logic, and presentation concerns:

Table 4: System Architecture Layers

Layer	Location	Responsibility
Model	python/models/	Query the database, call stored procedures and functions
View	python/gui/	Render the dark-mode GUI, display data to the user
Controller	python/gui/app.py	Handle navigation between pages and coordinate data flow
Database	MySQL(Aiven Cloud)	Store all data, enforce constraints, run triggers automatically

When a user records a new income or expense through the GUI, the application calls a Stored Procedure in the database. The procedure validates the transaction and inserts the record. A database Trigger then fires automatically to update the corresponding bank account balance and write an entry to the BalanceAuditLog table, all without any additional code in the Python application.

1.6 Key Features

Table 5: Key Features Implementation

Feature	Description	Implementation
User Authentication	Secure login and registration with hashed passwords	bcrypt + MySQL Users table
Income Management	Add and view income history with date and description	AddIncome procedure + Trigger
Expense Management	Add expenses by category with balance validation	AddExpense procedure + Trigger
Auto Balance Update	Bank balance updates automatically on every transaction	AfterIncomeInsert / AfterExpenseInsert triggers
Audit Trail	Full history of every balance change is recorded	BalanceAuditLog + AfterBalanceUpdate trigger
Financial Reports	Monthly income vs expense bar chart and category pie chart	Matplotlib + MonthlyFinancialSummary view
Budget Status	Real-time SURPLUS / BALANCED / DEFICIT indicator	GetBudgetStatus UDF
CSV Import	Seed database from CSV	csv_handler.py
Cloud Database	Database hosted on Aiven - accessible from anywhere	MySQL 8.4 on Aiven cloud

2 Database Analysis and Design

2.1 Data Model - ER Diagram

The system is designed around six core tables that capture all aspects of personal finance management. The Entity-Relationship diagram below illustrates the structure and relationships between these tables.

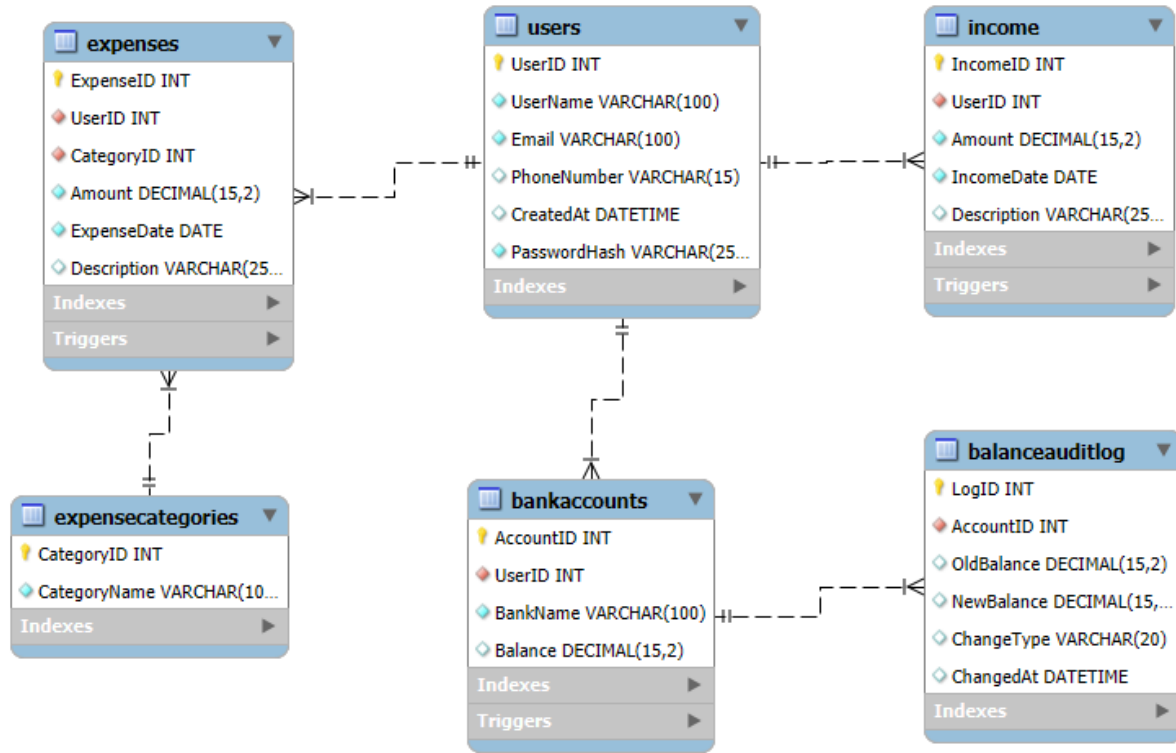


Figure 1: Entity-Relationship Diagram - Personal Finance Management System

The following table describes the role of each table in the system:

Table 6: Database Tables Description

Table	Description
Users	Stores user profile information, including name, email, phone number, and hashed password for authentication
BankAccounts	Tracks each user's bank accounts and their current balance, automatically updated by triggers
Income	Records all income transactions for each user, including amount, date, and description
ExpenseCategories	A lookup table containing predefined spending categories (e.g., Food, Transportation, Healthcare)
Expenses	Records all expense transactions, linked to both a user and a spending category
BalanceAuditLog	A bonus audit table that automatically logs every change to a bank account balance, including old and new values

2.2 Relational Schema

The relational schema defines how the six tables are connected through primary keys (PK) and foreign keys (FK). The diagram below shows the one-to-many relationships between tables:

```
Users (UserID PK)
|
|--< Income      (UserID FK) -- ON DELETE CASCADE
|--< Expenses    (UserID FK) -- ON DELETE CASCADE
|--< BankAccounts (UserID FK) -- ON DELETE CASCADE
      |
      |--< BalanceAuditLog (AccountID FK) -- ON DELETE CASCADE

ExpenseCategories (CategoryID PK)
|
|--< Expenses (CategoryID FK) -- ON DELETE RESTRICT
```

Each foreign key relationship is explained in the table below:

Table 7: Foreign Key Constraints and Behaviors

FK Column	Child Table	Parent Table	Behavior
UserID	Income	Users	ON DELETE CASCADE - deleting a user automatically removes all their income records
UserID	Expenses	Users	ON DELETE CASCADE - deleting a user automatically removes all their expense records
UserID	BankAccounts	Users	ON DELETE CASCADE - deleting a user automatically removes all their bank accounts
AccountID	BalanceAuditLog	BankAccounts	ON DELETE CASCADE - deleting an account removes its audit history
CategoryID	Expenses	ExpenseCategories	ON DELETE RESTRICT - prevents deleting a category that is still in use

The following screenshot from MySQL Workbench confirms the foreign key constraints as implemented in the database:

Table	Create Table
Expenses	<pre> CREATE TABLE "Expenses" ("ExpenseID" int NOT NULL AUTO_INCREMENT, "UserID" int NOT NULL, "CategoryID" int NOT NULL, "Amount" decimal(15,2) NOT NULL, "ExpenseDate" date NOT NULL, "Description" varchar(255) DEFAULT NULL, PRIMARY KEY ("ExpenseID"), KEY "UserID" ("UserID"), KEY "CategoryID" ("CategoryID"), KEY "idx_expense_date" ("ExpenseDate"), CONSTRAINT "Expenses_ibfk_1" FOREIGN KEY ("UserID") REFERENCES "Users" ("UserID") ON DELETE CASCADE, CONSTRAINT "Expenses_ibfk_2" FOREIGN KEY ("CategoryID") REFERENCES "ExpenseCategories" ("CategoryID") ON DELETE RESTRICT, CONSTRAINT "Expenses_chk_1" CHECK (('Amount' > 0))) </pre>

Figure 2: SHOW CREATE TABLE Expenses - Foreign Key Constraints

2.3 Data Constraints

Beyond foreign keys, additional constraints are applied to ensure data integrity across the system:

Table 8: Data Integrity Constraints

Constraint	Applied To	Purpose
CHECK (Amount > 0)	Income, Expenses	Prevents negative or zero values from being recorded as transaction amounts
UNIQUE (Email)	Users	Ensures each email address can only be registered once, preventing duplicate accounts
NOT NULL	UserName, Email, Amount, etc.	Ensures critical fields cannot be left empty
DEFAULT 0.00	BankAccounts.Balance	New bank accounts start with a balance of zero unless specified otherwise
DEFAULT CURRENT_TIMESTAMP	Users.CreatedAt, BalanceAudit-Log.ChangedAt	Automatically records the creation time without requiring manual input
DEFAULT ''	Users.PasswordHash	Provides a safe default value while requiring the application to always supply a hashed password

2.4 Sample Data

To demonstrate and test the system, sample data was loaded into the database via CSV import. The following screenshots show the contents of each table in MySQL Workbench after the initial data seeding.

Users Table

	UserID	UserName	Email	PhoneNumber	CreatedAt	PasswordHash
▶	1	Sophia Taylor	sophia.taylor@gmail.com	1-555-0301	2026-05-13 07:20:32	\$2b\$12\$w3DUPgBTmTX6rSdL4BeDDevRmj0sTa...
	2	Liam Johnson	liam.j.88@yahoo.com	1-555-0302	2026-05-13 07:20:35	\$2b\$12\$f6a9le8mf2GX44YecG2nc.MXdplMZtFD...
	3	Noah Williams	noah.wills@outlook.com	1-555-0303	2026-05-13 07:20:39	\$2b\$12\$CUIlTRGGgHq/8uj27LJZutrawNAP2Q...
	4	Olivia Brown	olivia_b@icloud.com	1-555-0304	2026-05-13 07:20:43	\$2b\$12\$hvNdKRa0FiT0vmC2rvo7XemzhegejY4c...
	5	Ethan Jones	ethan.jones.tech@gmail.com	1-555-0305	2026-05-13 07:20:46	\$2b\$12\$qsPshUhz6.INFMmx2OmrveJq4htrKNS1...
•	NULL	NULL	NULL	NULL	NULL	NULL

Figure 3: Sample data - Users table (5 records)

BankAccounts Table

	AccountID	UserID	BankName	Balance
▶	1	1	Chase Bank	21701.00
	2	2	Bank of America	13345.76
	3	3	Wells Fargo	30088.01
	4	4	Citibank	4769.76
	5	5	Capital One	29260.80
	6	6	US Bank	11568.00
•	NULL	NULL	NULL	NULL

Figure 4: Sample data - BankAccounts table (6 records)

Income Table

	IncomeID	UserID	Amount	IncomeDate	Description
▶	1	1	4500.00	2026-05-01	Monthly Salary
	2	2	150.00	2026-05-02	Sold old bicycle on Craigslist
	3	3	3200.00	2026-05-03	Freelance Web Development
	4	4	85.50	2026-05-04	Cash back reward from credit card
	5	5	5000.00	2026-05-05	Consulting Retainer
	6	6	1200.00	2026-05-06	Quarterly Dividend Payout
	7	7	4100.00	2026-05-07	Office Salary
	8	8	300.00	2026-05-08	Garage Sale profits
•	NULL	NULL	NULL	NULL	NULL

Figure 5: Sample data - Income table (8 records)

Expenses Table

	ExpenseID	UserID	CategoryID	Amount	ExpenseDate	Description
▶	1	1	1	12.50	2026-05-01	Iced Latte and pastry
	2	2	2	35.00	2026-05-02	Lyft ride to the airport
	3	3	3	85.00	2026-05-03	Monthly water bill
	4	4	4	15.99	2026-05-04	Spotify Family plan
	5	5	5	45.00	2026-05-05	Pharmacy vitamins
	6	6	6	120.00	2026-05-06	New running shoes
	7	7	7	65.00	2026-05-07	Haircut and styling
	8	8	1	45.00	2026-05-08	Sushi takeout
	9	9	2	28.00	2026-05-09	Gas station refill
	10	10	3	150.00	2026-05-10	Internet and cable bill
✱	NULL	NULL	NULL	NULL	NULL	NULL

Figure 6: Sample data - Expenses table (10 records)

ExpenseCategories Table

	CategoryID	CategoryName
▶	1	Food & Dining
	2	Transportation
	3	Utilities
	4	Entertainment
	5	Health & Fitness
	6	Shopping
	7	Personal Care
✱	NULL	NULL

Figure 7: Sample data - ExpenseCategories table

The sample dataset is summarized as follows:

Table 9: Sample Dataset Summary

Table	Record Count	Notes
Users	5	Loaded from <code>data/users.csv</code> with bcrypt-hashed passwords
BankAccounts	6	Loaded from <code>data/bank_accounts.csv</code> , 1–2 accounts per user
Income	8	Loaded from <code>data/income.csv</code> , distributed across all users
Expenses	10	Loaded from <code>data/expenses.csv</code> , covering 6 categories
ExpenseCategories	8	Pre-seeded via SQL script covering major spending categories

3 Advanced Database Implementation

This section describes the advanced database objects implemented in the system, explaining not only how each feature works, but why it was chosen and what concrete impact it has on system performance, reliability, and maintainability.

3.1 Indexes

Problem

Without indexes, every query that filters transactions by date or category requires MySQL to scan every row in the table sequentially. For a personal finance system that accumulates hundreds or thousands of transactions over time, this becomes increasingly slow, particularly for monthly report generation, which is the most frequent operation in the application.

Solution

Two indexes were manually created on the most commonly filtered columns:

Table 10: Database Indexes and Impact

Index	Column	Impact
idx_expense_date	Expenses(ExpenseDate)	Allows MySQL to jump directly to a date range instead of scanning all expense records when generating monthly reports
idx_income_date	Income(IncomeDate)	The same benefit applied to income queries, critical for the MonthlyFinancialSummary view

In addition, MySQL automatically created indexes on all Foreign Key columns (`UserID`, `CategoryID`, `AccountID`) to enforce referential integrity efficiently. These implicit indexes further accelerate JOIN operations between tables.

System Impact

With indexes in place, report queries that previously required full table scans now execute in sub-millisecond time regardless of table size. This is especially important for the dashboard, which reloads financial summaries every time the user navigates to a new page.

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
►	Expenses	0	PRIMARY	1	ExpenseID	A	50	NULL	NULL		BTREE		
	Expenses	1	UserID	1	UserID	A	20	NULL	NULL		BTREE		
	Expenses	1	CategoryID	1	CategoryID	A	7	NULL	NULL		BTREE		
	Expenses	1	idx_expense_date	1	ExpenseDate	A	50	NULL	NULL		BTREE		

Figure 8: All indexes on the Expenses table confirmed via `SHOW INDEX FROM Expenses LIMIT 5`

3.2 Views

Problem

Computing a monthly financial summary requires joining the `Income` and `Expenses` tables, grouping by user and month, and computing aggregates, a query that is both complex to write and easy to get wrong. If this logic is duplicated across multiple parts of the application, any future change requires updating every copy.

Solution

Two views encapsulate the most important aggregation queries, acting as *virtual tables* that the application can query as simply as `SELECT * FROM ViewName`:

MonthlyFinancialSummary - For each user and each month, this view computes total income, total expenses, and net savings. It uses a `UNION ALL` approach to combine income and expense rows before aggregating, ensuring that months with only income or only expenses are still correctly reported. This view is the backbone of both the dashboard stat cards and the bar chart in the Reports page.

CategoryWiseSpending - For each user, category, and month, this view sums all expense amounts. It drives the category pie chart in the Reports page, giving users a visual breakdown of their spending habits across categories such as Food, Transportation, and Healthcare.

System Impact

By centralizing reporting logic inside the database, the Python application remains simple and clean; it never needs to perform data aggregation itself. If the business logic changes (for example, adding a new way to calculate savings), only the view definition needs to be updated, with zero changes required in the application code.

	UserID	UserName	ReportYear	ReportMonth	TotalIncome	TotalExpense	NetSavings
▶	1	Sophia Taylor	2026	5	4800.00	54.50	4745.50
	2	Liam Johnson	2026	5	4950.00	145.00	4805.00
	3	Noah Williams	2026	5	4700.00	99.99	4600.01
	4	Olivia Brown	2026	5	335.50	40.99	294.51
	5	Ethan Jones	2026	5	9000.00	255.00	8745.00

Figure 9: Query result of `MonthlyFinancialSummary` for the sample dataset

	UserID	CategoryName	SpendingYear	SpendingMonth	TotalAmount
►	1	Food & Dining	2026	5	12.50
	2	Entertainment	2026	6	19.99
	3	Entertainment	2026	5	14.99
	4	Entertainment	2026	5	15.99
	8	Food & Dining	2026	5	45.00
	9	Entertainment	2026	6	40.00
	10	Entertainment	2026	5	60.00
	11	Entertainment	2026	5	25.00
	17	Entertainment	2026	6	12.00
	18	Entertainment	2026	5	55.00

Figure 10: Query result of `CategoryWiseSpending` for the sample dataset

3.3 User Defined Functions (UDFs)

Problem

Several parts of the system need to compute the same values repeatedly: total income for a month, total expenses for a month, and whether the user is in surplus or deficit. Without functions, this logic would need to be duplicated across views, procedures, and application queries.

Solution

Three scalar functions were implemented, each accepting a (`UserID`, `Month`, `Year`) parameter set:

Table 11: User Defined Functions (UDFs) and Roles

Function	Role in the System
<code>GetTotalIncome</code>	Returns the sum of all income for a user in a given month. Used internally by <code>GetBudgetStatus</code> and directly by the Python dashboard
<code>GetTotalExpenses</code>	Returns the sum of all expenses for a user in a given month. Used internally by <code>GetBudgetStatus</code> and the monthly closure procedure
<code>GetBudgetStatus</code>	Calls the two functions above, computes the net difference, and returns <code>SURPLUS</code> , <code>BALANCED</code> , or <code>DEFICIT</code> . Powers the real-time budget badge on the application dashboard

System Impact

Functions enforce the DRY principle (Don't Repeat Yourself) at the database level. `GetBudgetStatus` in particular allows the Python application to determine a user's fi-

financial health with a single function call, rather than performing multiple queries and comparisons in application code. This also means the budget logic can be tested and verified directly in SQL, independently of the application layer.

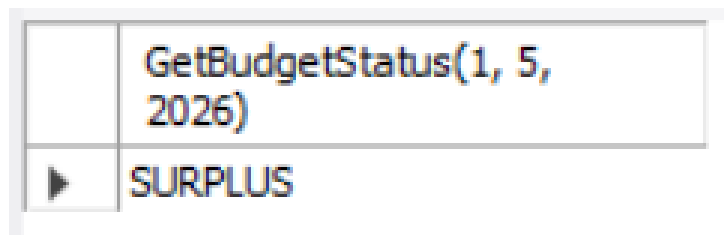


Figure 11: `SELECT GetBudgetStatus(1, 5, 2026)` returning `SURPLUS` for User 1 in May 2026

3.4 Triggers

Problem

Every time a transaction is recorded, the corresponding bank account balance must be updated, and the change must be logged for auditing. If this responsibility is left to the application layer, there is a risk of inconsistency, a bug, a crash, or a direct database `INSERT` that bypasses the application, which would leave balances out of sync.

Solution

Three triggers automate the entire process at the database level, making it impossible for a transaction to be recorded without the balance being updated:

Table 12: Database Triggers and Roles

Trigger	Role
<code>AfterIncomeInsert</code>	Fires after every <code>INSERT</code> on the <code>Income</code> table. Automatically adds the income amount to the user's primary bank account balance
<code>AfterExpenseInsert</code>	Fires after every <code>INSERT</code> on the <code>Expenses</code> table. Automatically subtracts the expense amount from the user's primary bank account balance
<code>AfterBalanceUpdate</code>	Fires after every <code>UPDATE</code> on <code>BankAccounts</code> . Writes a record to <code>BalanceAuditLog</code> containing the old balance, new balance, change type, and timestamp

The complete automation chain when a user adds an income transaction is illustrated below:

```
[User submits income in GUI]
|
v
INSERT into Income table
|
```

```

    v    (automatic)
AfterIncomeInsert trigger fires
    |
    v
UPDATE BankAccounts SET Balance = Balance + amount
    |
    v    (automatic)
AfterBalanceUpdate trigger fires
    |
    v
INSERT into BalanceAuditLog
(old balance, new balance, 'INCOME', timestamp)

```

System Impact

This design guarantees **data consistency by construction**; the application never needs to manually update balances or write audit records. Even if a developer inserts a transaction directly via SQL Workbench, the triggers will still fire and keep all data synchronized. The audit log also provides full traceability, allowing reconstruction of the complete balance history for any account at any point in time.

	LogID	AccountID	OldBalance	NewBalance	ChangeType	ChangedAt
▶	1	1	12500.50	17000.50	INCOME	2026-05-13 07:22:10
	2	2	8400.75	8550.75	INCOME	2026-05-13 07:22:11
	3	3	22300.00	25500.00	INCOME	2026-05-13 07:22:12
	4	4	4500.25	4585.75	INCOME	2026-05-13 07:22:13
	5	5	15600.80	20600.80	INCOME	2026-05-13 07:22:14
	6	6	9200.00	10400.00	INCOME	2026-05-13 07:22:15
	7	7	3400.50	7500.50	INCOME	2026-05-13 07:22:16
	8	8	45000.00	45300.00	INCOME	2026-05-13 07:22:18
	9	9	11200.75	13300.75	INCOME	2026-05-13 07:22:19
	10	10	7800.00	8400.00	INCOME	2026-05-13 07:22:20
•	NULL	NULL	NULL	NULL	NULL	NULL

Figure 12: BalanceAuditLog automatically populated after several income and expense transactions

3.5 Stored Procedures

Problem

Adding an income or expense record is not a single-step operation; ion, it involves validation, insertion, and returning meaningful feedback to the user. If this logic lives in the application layer, it creates a tight coupling between the Python code and the database schema, making the system harder to maintain and test.

Solution

Three stored procedures encapsulate the core business operations:

AddIncome - Validates that the amount is positive, inserts the record, and returns a success message with the new record ID. The balance update is handled automatically by **AfterIncomeInsert**.

AddExpense - The most critical procedure. Before inserting, it retrieves the user's current balance and raises a database-level error if the balance is insufficient. This prevents overdrafts regardless of how the data is accessed - through the GUI, CLI, or direct SQL. On success, it returns the remaining balance to the application for display to the user.

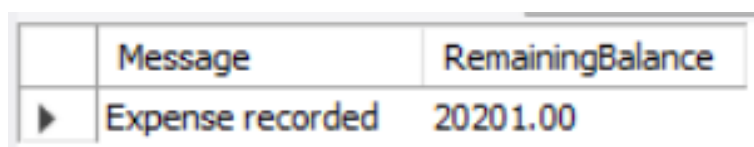
MonthlyClosure - Generates a closing balance report for all of a user's bank accounts, intended for end-of-month financial review.

The logic flow of **AddExpense** is summarized below:

1. Retrieve current balance from **BankAccounts**
2. **If** balance < requested amount → raise error *"Insufficient account balance"*
3. **Else** insert record into **Expenses**
4. Return confirmation message and remaining balance

System Impact

By moving validation logic into stored procedures, the Python application becomes simpler and more robust. The **AddExpense** procedure, in particular, acts as a *database-enforced business rule*; it is impossible to record an expense that exceeds the available balance, no matter which interface is used to access the database.



	Message	RemainingBalance
▶	Expense recorded	20201.00

Figure 13: Result of calling `CALL AddExpense(1, 1, 1500, '2026-05-13', 'Dinner')` - procedure returns confirmation and remaining balance

3.6 Database Security and Access Control

Problem

A common security mistake in student projects is connecting the application directly to the database using the `root` account. This gives the application unlimited privileges, including `DROP TABLE`, `DELETE`, and schema modification, creating a serious risk if credentials are ever exposed or the application is compromised.

Solution

Following the **principle of least privilege**, two dedicated MySQL users were created, each with only the permissions strictly necessary for their role:

Table 13: Database Security Users and Permissions

User	Permissions	Purpose and Rationale
<code>finance_app_user</code>	SELECT, INSERT, UPDATE	Used by the Python application for all normal operations. Cannot DELETE records or modify the schema, limiting the blast radius of any potential security incident
<code>finance_viewer</code>	SELECT only	Intended for read-only reporting access. Cannot modify any data under any circumstances

System Impact

This separation of privileges provides two layers of protection. First, it limits accidental damage; a bug in the application cannot accidentally delete records because the connected user lacks DELETE permission. Second, it limits the impact of a security breach, even if `finance_app_user` credentials are compromised, an attacker cannot drop tables or modify the database schema. The `root` account is only used during initial setup and is never referenced in any application configuration file. It is worth noting that in the current cloud deployment on Aiven, the `finance_app_user` account is used by the application, while `avnadmin` serves the equivalent role of `root` for administrative tasks only.

4 Python Application Development

This section describes the design and implementation of the Python application that connects to the MySQL database and provides users with a full-featured personal finance management interface.

4.1 Application Architecture

The application follows a simplified **MVC (Model-View-Controller)** pattern, clearly separating data access, presentation, and navigation concerns into distinct layers. This structure makes the codebase easier to maintain; a change in the database schema only requires updating the `models/` layer, while a UI change only affects `gui/`.

Table 14: Python Application Architecture

Layer	Location	Responsibility
Model	<code>models/</code>	Communicates directly with the database - executes queries, calls stored procedures, and returns structured data to the View layer
View	<code>gui/</code>	Renders all GUI components - pages, forms, tables, and charts. Never accesses the database directly
Controller	<code>gui/app.py</code>	Manages navigation between pages, holds the current user session, and coordinates data flow between Model and View

The project directory structure is organized as follows:

```
python/
+-- main.py           Entry point
+-- config.py         Database connection
+-- database.py       Core DB access functions
+-- ca.pem            SSL certificate for Aiven cloud DB
+-- SQLLedger.spec    PyInstaller build configuration
+-- models/
|   +-- user.py       --> Users table operations
|   +-- income.py     --> Income table operations
|   +-- expense.py    --> Expenses table operations
|   +-- account.py    --> BankAccounts table operations
+-- gui/
|   +-- app.py        --> Controller: session + navigation
|   +-- theme.py      --> Colour palette + reusable widgets
|   +-- login_window.py --> Login and registration window
|   +-- base_page.py  --> Base class for all pages
|   +-- pages/
|       +-- dashboard.py --> Dashboard page
|       +-- incomes.py   --> Incomes page
|       +-- expenses.py  --> Expenses page
|       +-- report.py    --> Reports and charts page
|       +-- account.py   --> Bank account page
```

```

|      +-- users.py      --> User management page
+-- reports/
|      +-- charts.py     --> Standalone chart generation functions
|                          (bar chart, pie chart) exported as PNG
+-- utils/
|      +-- csv_handler.py --> CSV import and export
+-- data/
    +-- users.csv
    +-- bank_accounts.csv
    +-- income.csv
    +-- expenses.csv

```

4.2 Database Connection Layer

All database interactions are centralized in `database.py`, which exposes four functions used throughout the application. No other file connects to MySQL directly; this ensures that the connection configuration is defined in exactly one place and can be changed without touching any other file.

Table 15: Database Connection Layer Functions

Function	Purpose
<code>fetch_all(query, params)</code>	Executes a SELECT query and returns all matching rows as a list of dictionaries
<code>fetch_one(query, params)</code>	Executes a SELECT query and returns a single row as a dictionary - used for login and user lookups
<code>execute(query, params)</code>	Executes an INSERT, UPDATE, or DELETE statement and returns the ID of the last inserted row
<code>call_procedure(name, params)</code>	Calls a MySQL stored procedure by name and returns all result sets - used for AddIncome and AddExpense

The connection itself is established via `mysql-connector-python` using the credentials stored in `config.py`. The application always connects as `finance_app_user` - never as `root`, ensuring that the principle of least privilege is enforced at the application level.

```

# config.py
DB_CONFIG = {
    "host":      "<aiven-host>",
    "port":      <port>,
    "user":      "finance_app_user",
    "password":  "<password>",
    "database":  "PersonalFinanceDB",
    "ssl_ca":    "ca.pem"
}

```

```

# database.py - core connection
def get_connection():
    return mysql.connector.connect(**DB_CONFIG)

# Calling a stored procedure
def call_procedure(proc_name, params):
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.callproc(proc_name, params)
    results = []
    for result in cursor.stored_results():
        results.extend(result.fetchall())
    return results

```

4.3 Graphical User Interface (GUI)

The primary interface of the application is a desktop GUI built with **Tkinter**, Python’s built-in GUI framework. The design follows a **dark mode** aesthetic with a fixed sidebar for navigation and a dynamic content area on the right that switches between six pages.

The application is structured so that all five pages are created once at login and stacked on top of each other. Navigation works by calling `page.lift()` to bring the selected page to the front, and `page.refresh()` to reload its data from the database, ensuring the user always sees up-to-date information.

Login & Registration

The application opens with a login window requiring an email address and password. New users can navigate to the registration form by clicking “*Create one*”.

Personal Finance Manager - Login

Finance Manager
Sign in to your account

Email
Enter your email

Password

Login

Don't have an account? [Create one](#)

Figure 14: Login window, email, and password authentication

The registration form collects the user's full name, email, phone number, password (with confirmation), and optionally a bank name and initial balance to set up their first account immediately upon registration.

Personal Finance Manager - Login

Create Account
Fill in your details below

Full Name
Enter your full name

Email
Enter your email

Phone
Phone number (optional)

Password

Confirm Password

Bank Name
Enter your bank name

Initial Balance
Enter initial balance (e.g. 1000)

Create Account

Figure 15: Login window, email, and password authentication

Passwords are never stored in plain text; they are hashed using **bcrypt** before being

saved to the database, and verified using `bcrypt.checkpw()` at login time. If the two password fields do not match, the form displays an error and prevents submission.

Dashboard

The dashboard is the first page the user sees after logging in. It provides an at-a-glance summary of the current month's financial status through `fstats` stat cards (Total Income, Total Expense, Net Savings, Total Balance), a real-time budget status badge (SURPLUS / BALANCED / DEFICIT), a recent transactions table, and an embedded bar chart comparing income and expenses over recent months.

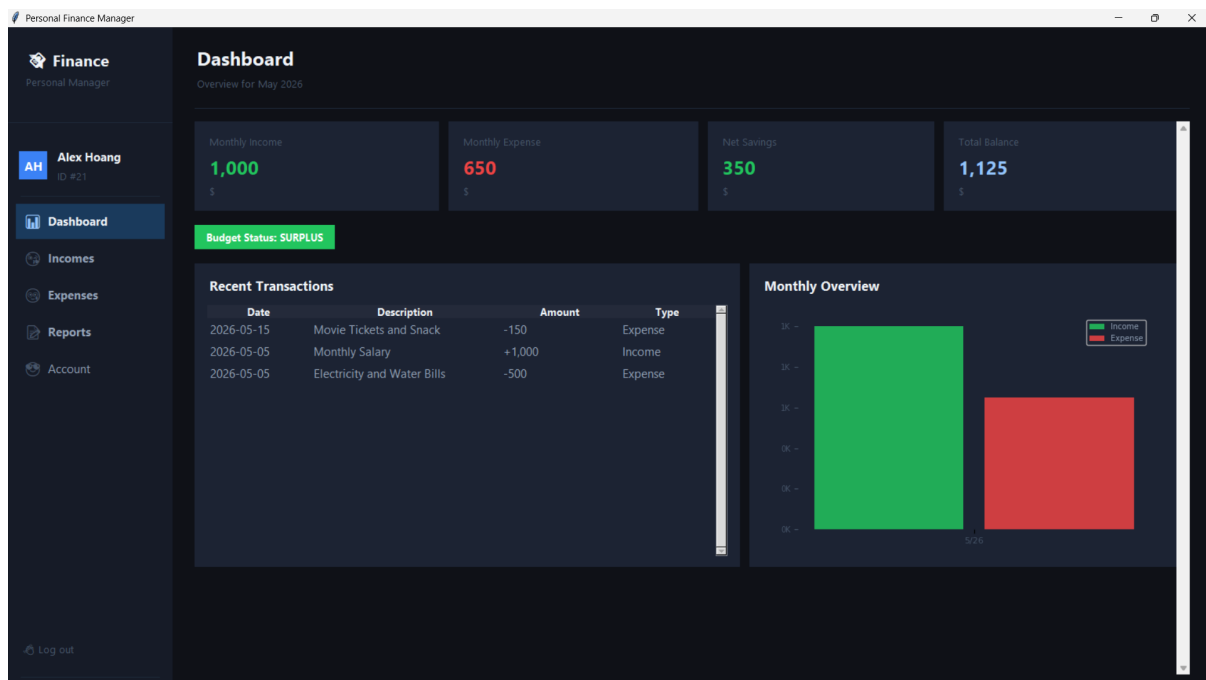


Figure 16: Dashboard - stat cards, budget status, recent transactions, and bar chart

Income Page

The Income page is split into two columns: a form on the left for adding new income entries, and a scrollable history table on the right. When a new income record is submitted, the application calls the `AddIncome` stored procedure, which inserts the record and triggers an automatic balance update. The dashboard and accounts pages are then refreshed automatically to reflect the new balance.

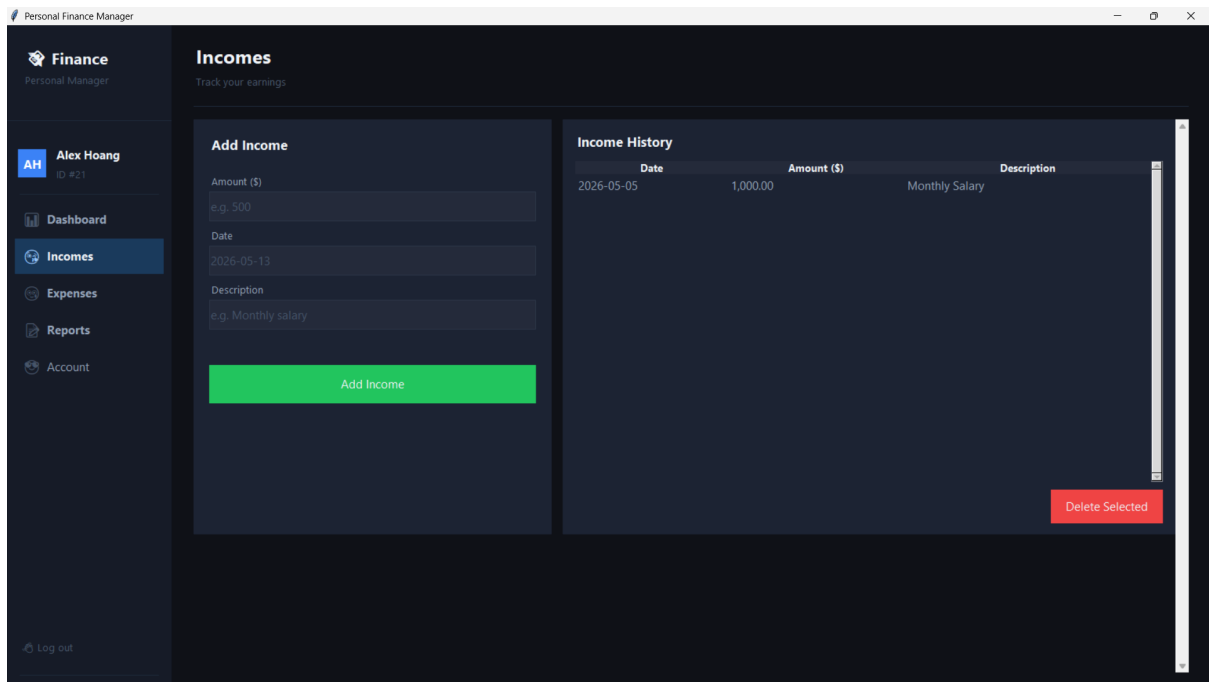


Figure 17: Income page - entry form and transaction history

Expenses Page

The Expenses page follows the same two-column layout. The form includes a dropdown menu populated from the `ExpenseCategories` table, ensuring that all expenses are consistently categorized. Before inserting, the `AddExpense` procedure checks whether the user's balance is sufficient - if not, an error message is displayed and no record is created.

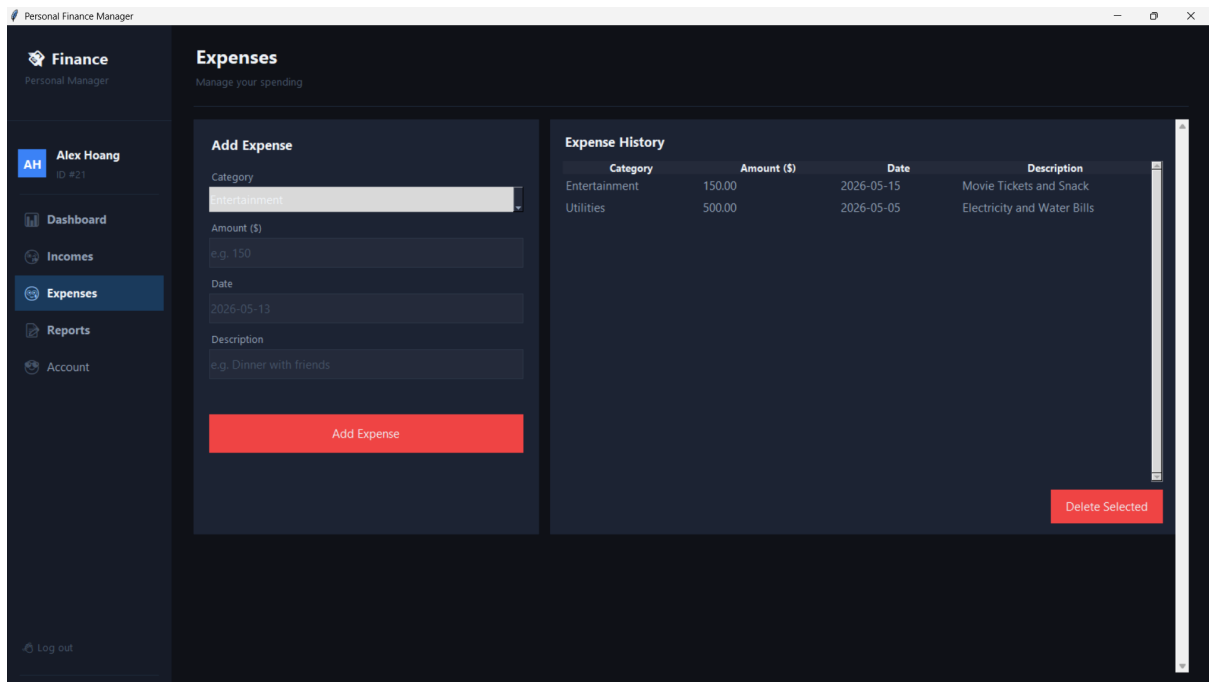


Figure 18: Expenses page - category dropdown, entry form, and history table

Reports Page

The Reports page presents two embedded Matplotlib charts: a grouped bar chart comparing monthly income and expenses side by side, and a pie chart showing spending distribution across categories. Both charts query data from the `MonthlyFinancialSummary` and `CategoryWiseSpending` database views, and are re-rendered every time the page is opened to ensure they reflect the latest data.

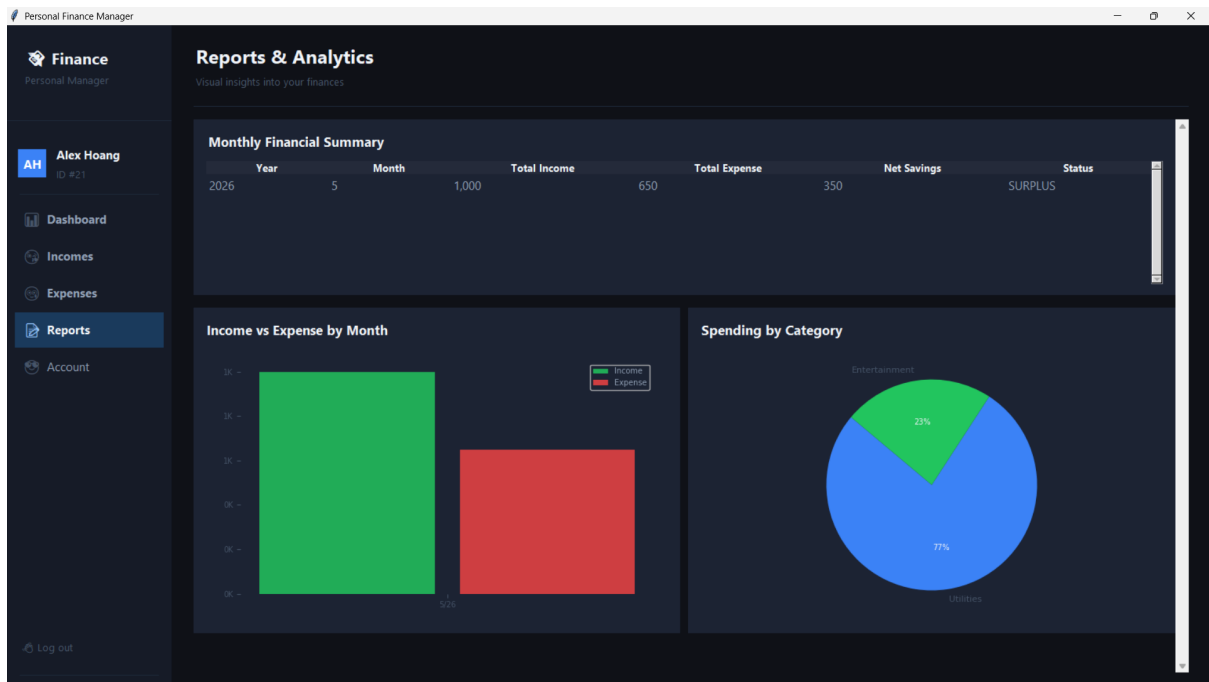


Figure 19: Reports page - monthly bar chart and category spending pie chart

Accounts Page

The Accounts page displays all bank accounts belonging to the logged-in user, showing each account's name and current balance, followed by the complete **BalanceAuditLog** for that user. This page demonstrates the trigger chain in action - every income or expense transaction results in a new audit log entry appearing here automatically, with no manual intervention required.

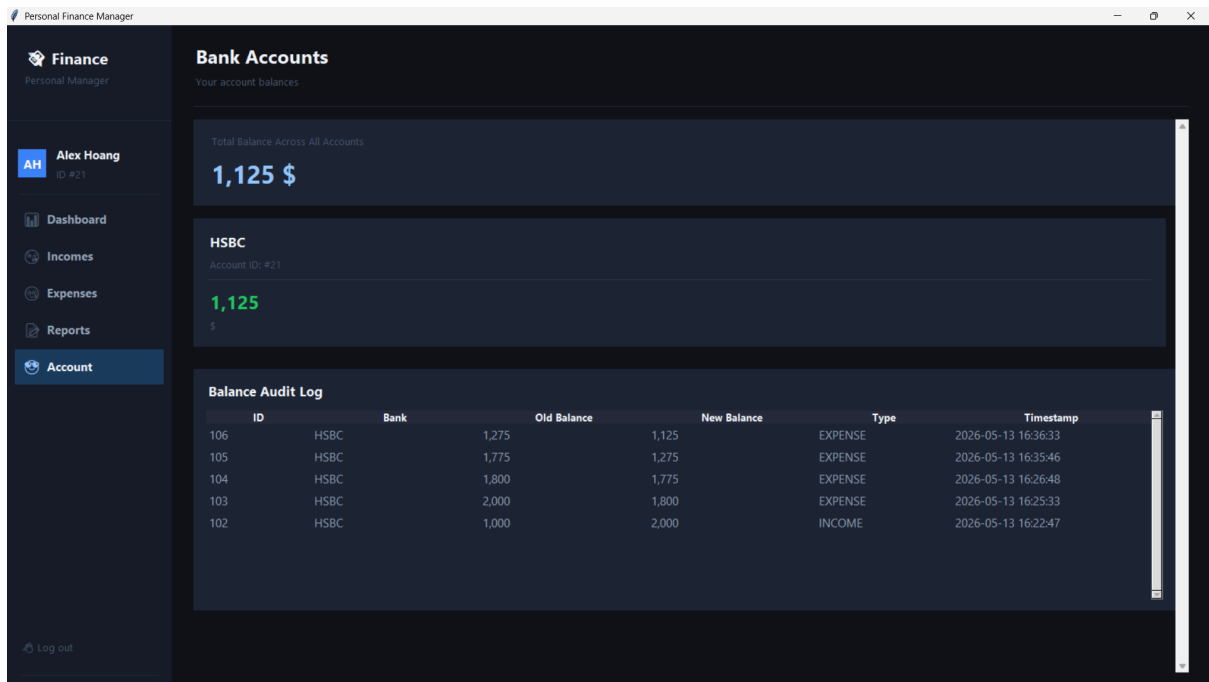


Figure 20: Accounts page - account balance cards and balance audit log

4.4 Reporting and Data Visualization

All charts in the application are generated using **Matplotlib** and embedded directly into the Tkinter window using **FigureCanvasTkAgg** - a backend adapter that renders a Matplotlib figure as a native Tkinter widget. This means charts are fully interactive within the application window, with no external browser or file needed.

Monthly Income vs Expense Bar Chart

This chart groups transactions by month and renders two bars per period - one for total income (green) and one for total expenses (red). It allows users to identify months with unusually high spending or unusually low income at a glance. The Y-axis values are automatically formatted in millions (e.g. *15M*) to keep labels readable.

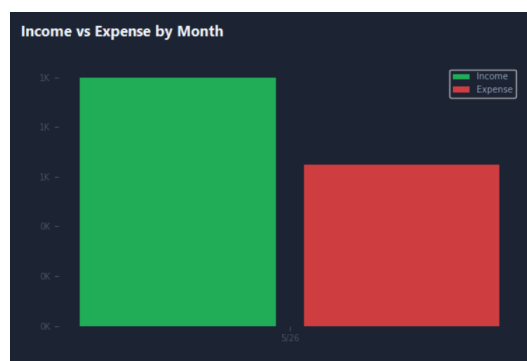


Figure 21: Monthly Income vs Expense bar chart embedded in the Reports page

Category Spending Pie Chart

This chart shows the proportion of total spending allocated to each expense category (Food, Transportation, Healthcare, etc.), queried directly from the `CategoryWiseSpending` view. It helps users understand their spending habits and identify categories where they may be overspending.

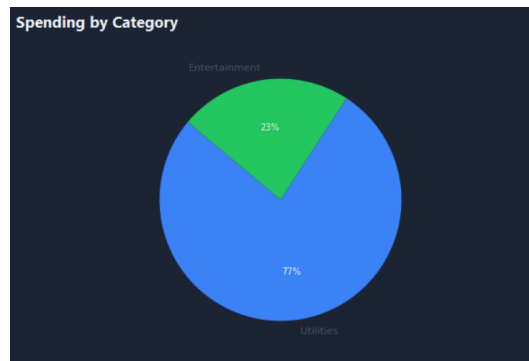


Figure 22: Category spending pie chart embedded in the Reports page

Real-time Balance Update

A key feature of the application is that bank account balances update in real time after every transaction - without any manual refresh. This is achieved through a combination of database triggers and application-level page refreshing:

1. User submits a transaction via the Income or Expenses form
2. The application calls the appropriate stored procedure
3. The database trigger fires automatically and updates `BankAccounts`
4. The application calls `refresh()` on the Dashboard and Accounts pages
5. The updated balance is immediately visible across all pages

This design ensures that the displayed balance is always consistent with the database state, and that no additional user action is required to see the effect of a transaction.

5 Conclusion and Recommendation

5.1 Achievements and Key Deliverables

The project successfully implemented a comprehensive Personal Financial Management System (PFMS) that integrates a Python-based Graphical User Interface (GUI) with a robust MySQL relational database. The following table summarizes the key requirements and the achieved outcomes:

Table 16: Project Achievements Summary

Category	Feature	Implementation Details
Core Operations	Income & Expense Management	Full transaction management (Add, View, Delete) implemented for income and expense records
Financial Logic	Financial Summary	Real-time balance calculations handled by Database Views and Scalar Functions
Budget Management	Budget Alerts	Automated status warnings integrated via the <code>GetBudgetStatus</code> function
Automation	Automatic Triggers	Implementation of 3 database triggers for real-time data synchronization
Database Logic	Advanced Data Handling	Built 3 modular Stored Procedures to process complex transactions
Security	Role-Based Access Control	Implemented 2 distinct roles (Admin/User) to restrict and manage system access
User Interface	Python Desktop App	Modern Graphical User Interface (GUI) developed using Tkinter
Data Visualization	Analytical Reports	Interactive Bar and Pie charts generated for spending and income analysis
Data Integrity	Audit Trail (Bonus)	Change tracking and historical logging implemented via the <code>BalanceAuditLog</code> table

5.2 Future Developments

While the current system fulfills all core requirements, several enhancements are proposed to evolve the application into a more sophisticated financial tool.

Short-term Improvements

In the immediate future, the following features can be integrated to enhance the user experience:

- **Savings Goal Module:** Implementation of a savings tracking system with visual progress bars to help users monitor specific financial targets.

- **Budget Threshold Notifications:** Advanced category-specific limits that trigger visual warnings within the GUI as users approach their spending capacity.
- **PDF Report Generation:** Integration of the `ReportLab` library to enable users to export monthly financial summaries and transaction histories into professional PDF documents.

Long-term Academic and Technical Goals

Looking further ahead, the project proposes a transition from a management tool to an intelligent financial assistant:

- **Predictive Financial Analytics:** Leveraging Machine Learning libraries such as `scikit-learn` to analyze historical patterns and forecast future spending behaviors.
- **Open Banking Integration:** Developing API connections with regional payment gateways (e.g., VietQR, MoMo) to allow real-time, automated transaction synchronization.
- **Multi-currency Support:** Expanding the database schema to handle multiple currencies with automated exchange rate conversion, catering to international financial activities.

References

- [1] MySQL 8.4 Reference Manual. *Oracle Corporation*, 2024. <https://dev.mysql.com/doc/refman/8.4/en/>
- [2] MySQL Connector/Python Developer Guide. *Oracle Corporation*, 2024. <https://dev.mysql.com/doc/connector-python/en/>
- [3] Python Software Foundation. *tkinter - Python interface to Tcl/Tk*, 2024. <https://docs.python.org/3/library/tkinter.html>
- [4] J. D. Hunter. *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3):90-95, 2007. <https://matplotlib.org/stable/>
- [5] bcrypt - Modern password hashing for your software. <https://pypi.org/project/bcrypt/>
- [6] Aiven for MySQL Documentation. <https://aiven.io/docs/products/mysql>